Tài liệu mã nguồn `hds_touchsim`

Mẫu pin HDS 5×3 → kích thích TouchSim → số spike → độ tương đồng

07/11/2026

Tài liệu này mô tả những gì mã nguồn thực hiện: từng module, từng hàm, tham số, giá trị trả về, công thức và thuật toán bên trong, cùng cách mã được kiểm thử. Chỉ phần được lập trình được đề cập.

Mục lục

1 Tổng quan mã nguồn **2**

1.1 Cấu trúc tệp . . . 2

1.2 Phụ thuộc giữa các module . . . 2

1.3 Hai hệ toạ độ trong mã . . . 2

2 Module `hds.py` **3**

2.1 Hằng số và `HDSSpec` . . . 3

2.2 Mẫu chữ số . . . 3

2.3 Hình học . . . 4

2.4 `fit_to_region` . . . 4

3 Module `adapter.py` **4**

3.1 Hằng số . . . 4

3.2 Chuẩn hoá kích thước tay . . . 5

3.3 Đổi vị trí pin . . . 5

3.4 `region_mask_fn` . . . 5

3.5 `build_stimulus` . . . 6

3.6 `balance_classes` . . . 6

3.7 `make_population` . . . 7

3.8 `respond` . . . 8

4 Module `evaluate.py` **8**

5 Module `viz.py` **8**

6 Script `scripts/run_pipeline.py` 9

7 Kiểm thử 9

7.1 `tests/test_hds.py` (không cần TouchSim) 9

7.2 `tests/test_adapter.py` 10

7.3 `tests/fake_touchsim.py` 10

8 Notebook `hds_touchsim_clean.ipynb` 10

9 Ghi chú về hành vi của mã 10

1 Tổng quan mã nguồn

1.1 Cấu trúc tệp

Tệp	Vai trò
`hds_touchsim/hds.py`	Thông số HDS (`HDSSpec`), mẫu chữ số, hình học pin, tìm vị trí đặt HDS. Chỉ dùng `numpy`.
`hds_touchsim/adapter.py`	Cầu nối với TouchSim: chuẩn hoá kích thước tay, đổi toạ độ, dựng kích thích, tạo và cân bằng quần thể afferent.
`hds_touchsim/evaluate.py`	Số spike, hệ số Pearson, ma trận tương đồng, tổng spike theo lớp.
`hds_touchsim/viz.py`	Một hàm vẽ: đường viền tay và HDS.
`hds_touchsim/__init__.py`	Xuất các tên công khai của package.
`scripts/run_pipeline.py`	Chạy cho 10 chữ số và lưu kết quả.
`tests/`	`test_hds.py`, `test_adapter.py`, `fake_touchsim.py`, `conftest.py`.
`hds_touchsim_clean.ipynb`	Cùng mã trong một notebook.

1.2 Phụ thuộc giữa các module

`hds.py` không import module nào khác của package. `adapter.py` import `HDSSpec`, `bounding_box_px`, `pin_positions_px` từ `hds.py`. `evaluate.py` chỉ dùng `numpy`. `viz.py` import từ `adapter.py` và `hds.py`. Thư viện `touchsim` *không* được import ở đầu `adapter.py`: mọi hàm nhận module `touchsim` qua tham số `ts`, và `load_touchsim()` import nó khi cần. Nhờ vậy `hds.py`, `evaluate.py` và phần lớn `adapter.py` dùng được khi chưa cài TouchSim.

1.3 Hai hệ toạ độ trong mã

- **pixel**: toạ độ trên ảnh bàn tay của TouchSim. Các hàm có hậu tố `_px` làm việc trong hệ này.
- **hand**: toạ độ mm của TouchSim, là toạ độ mà `ts.stim_ramp` và vị trí afferent sử dụng (`pins_hand_mm`, `pop.location`).

Chuyển đổi qua `ts.hand_surface.pixel2hand` và `ts.hand_surface.hand2pixel`.

2 Module `hds.py`

2.1 Hằng số và `HDSSpec`

`ROWS, COLS = 5, 3`. `HDSSpec` là `dataclass(frozen=True)` (bất biến) gồm các trường:

Trường	Mặc định	Ý nghĩa
`pin_diameter_mm`	10.0	đường kính pin
`pitch_mm`	15.0	khoảng cách giữa hai tâm pin liền kề
`depth_mm`	1.0	độ sâu đẩy, truyền làm `amp` cho `stim_ramp`
`pressure_mpa`	0.3	được khai báo nhưng không mã nào đọc
`ramp_s`	0.2	thời gian ramp lên và ramp xuống
`hold_s`	2.6	thời gian giữ
`width_mm`, `height_mm`	50.0, 80.0	kích thước hộp bao
`rows`, `cols`	5, 3	số hàng, số cột pin

`HDSSpec.duration_s -> float`

Thuộc tính, bằng $2 \cdot$ `ramp_s` + `hold_s` (mặc định $0{,}2 + 2{,}6 + 0{,}2 = 3{,}0$ s).

`HDSSpec.pin_offsets_mm() -> ndarray (rows, cols, 2)`

Trả về toạ độ tâm từng pin so với tâm HDS, đơn vị mm, trục x sang phải, trục y hướng lên, hàng 0 ở trên cùng:

$$x_j = \Big(j - \tfrac{C-1}{2}\Big)p, \qquad y_i = \Big(\tfrac{R-1}{2} - i\Big)p. \tag{1}$$

Với $R = 5$, $C = 3$, $p = 15$: $x \in \{-15, 0, 15\}$, $y \in \{30, 15, 0, -15, -30\}$. Mã dùng `np.meshgrid(xs, ys)` rồi `np.stack([gx, gy], axis=-1)`.

2.2 Mẫu chữ số

`_GLYPHS` là từ điển chữ số → danh sách 5 chuỗi, mỗi chuỗi 3 ký tự `0/1`. Ví dụ chữ số 3: `["111","001","111","001","111"]`.

`digit_tensor(d: int) -> ndarray uint8 (5, 3)`

Đổi từng ký tự của glyph thành số nguyên; 1 nghĩa là pin hoạt động. Số pin hoạt động của chữ số 0–9 là $12, 5, 11, 11, 9, 11, 10, 7, 13, 10$.

`hamming_matrix() -> ndarray int (10, 10)`

Phần tử (a, b) là số ô khác nhau giữa `digit_tensor(a)` và `digit_tensor(b)`:

$$H_{ab} = \sum_{i,j} \mathbb{1}[T_a(i,j) \neq T_b(i,j)]\,. \tag{2}$$

Ma trận đối xứng, đường chéo bằng 0. Hai cặp có $H = 1$ là $(0, 8)$ và $(4, 9)$; $H_{01} = 13$ là lớn nhất.

2.3 Hình học

`pin_positions_px(spec, center_px, pxl_per_mm=1.0) -> ndarray (rows, cols, 2)`

$$\mathbf{P}_{ij} = \mathbf{c} + \rho\,\mathbf{o}_{ij}, \tag{3}$$

với $\mathbf{c}$ là `center_px`, $\mathbf{o}_{ij}$ là `pin_offsets_mm()` và $\rho =$ `pxl_per_mm`. Độ lệch y được cộng trực tiếp vào toạ độ pixel, không đảo dấu.

`bounding_box_px(spec, center_px, pxl_per_mm=1.0) -> (xmin, ymin, xmax, ymax)`

Hộp $\left[c_x \mp \frac{W}{2}\rho,\ c_y \mp \frac{H}{2}\rho\right]$ với W, H là `width_mm`, `height_mm`.

2.4 `fit_to_region`

`fit_to_region(spec, region_mask_fn, search_box, pxl_per_mm=1.0, step_px=5) -> (best_center, n_inside, grid)`

Tìm tâm HDS sao cho nhiều pin nhất có tâm nằm trong một vùng. Tham số:

- `region_mask_fn(x, y) -> bool`: cho biết điểm pixel (x, y) có thuộc vùng không.
- `search_box=(xmin, xmax, ymin, ymax)`: hộp chứa các tâm ứng viên (cả hai đầu mút đều được xét, vì dùng `np.arange(min, max+1, step_px)`).

Thuật toán:

1. Với mỗi tâm ứng viên (c_x, c_y) trên lưới, tính 15 vị trí pin bằng `pin_positions_px`, đếm số pin mà `region_mask_fn` trả `True`, ghi vào từ điển `grid[(cx, cy)]`.
2. Lấy $n^* = \max$ của `grid`.
3. Lấy tập các tâm có đúng n^* pin (đồng hạng), tính trọng tâm của tập này, chọn tâm trong tập có khoảng cách bình phương tới trọng tâm nhỏ nhất. Mục đích: tránh chọn tâm sát mép vùng.

Giá trị trả về: tâm được chọn (tuple), n^*, và toàn bộ `grid` để xem lại. Số lần đánh giá là (số tâm trên lưới)$\times$15. Với hộp mặc định của `run_pipeline.py` ($x \in [40, 400]$, $y \in [100, 520]$, bước 4) lưới có $91 \times 106 = 9646$ tâm.

3 Module `adapter.py`

3.1 Hằng số

- `PALM_REGIONS = ["Pw1_p","Pw2_p","Pw3_p","Pw4_p","Pp1_p","Pp2_p"]`: tên sáu vùng lòng bàn tay trong `hand_surface.tags`.
- `REAL_HAND_LENGTH_MM = 189.0`: chiều dài tay đích.
- `CLASS_RATIO = {"SA1": 2, "RA": 4, "PC": 1}`: tỉ lệ số afferent mục tiêu (SA1 gấp đôi PC, RA gấp đôi SA1, theo [1]).

3.2 Chuẩn hoá kích thước tay

`load_touchsim() -> module`

Import và trả về `touchsim`.

`ts_px_per_mm(ts) -> float`

Trả về `ts.hand_surface.pxl_per_mm` ($\rho_{ts} \approx 2{,}18$ px/mm). Mã chỉ đọc giá trị này, không ghi đè.

`ts_hand_length_mm(ts) -> float`

Ghép mọi đa giác trong `hand_surface.boundary`, chia cho ρ_{ts} và lấy `np.ptp` theo trục y:

$$L_{ts} = \max_y - \min_y \quad (\text{đơn vị mm TouchSim}, \ \approx 209). \tag{4}$$

`hand_scale(ts) -> float`

$$s_h = \frac{\texttt{REAL_HAND_LENGTH_MM}}{L_{ts}} = \frac{189}{L_{ts}} \approx 0{,}904. \tag{5}$$

`px_per_mm(ts) -> float`

$$\rho = \frac{\rho_{ts}}{s_h} \approx 2{,}41 \text{ pixel trên mỗi mm thật}. \tag{6}$$

Đây là hệ số được dùng khi đặt HDS lên ảnh: ảnh bàn tay không đổi, còn 1 mm thật ứng với ρ pixel. Kích thước HDS luôn tính theo mm thật (50×80 mm $\approx 121 \times 193$ px), không nhân với s_h.

3.3 Đổi vị trí pin

`pins_px(ts, spec, center_px) -> ndarray (rows, cols, 2)`

`pin_positions_px(spec, center_px, px_per_mm(ts))`.

`pins_hand_mm(ts, spec, center_px) -> ndarray (rows, cols, 2)`

Làm phẳng kết quả của `pins_px` thành $(15, 2)$, gọi `hand_surface.pixel2hand`, rồi đưa về $(rows, cols, 2)$. Vì `pixel2hand` chia khoảng cách pixel cho ρ_{ts} còn khoảng cách pixel giữa hai pin là $\rho \cdot p = \rho_{ts} p / s_h$, nên khoảng cách giữa hai pin trong hệ hand là

$$p_{ts} = \frac{p}{s_h} \approx 16{,}6 \text{ mm}, \tag{7}$$

và hộp HDS là khoảng $55{,}3 \times 88{,}5$ mm trong đơn vị TouchSim.

3.4 `region_mask_fn`

`region_mask_fn(ts, tags=PALM_REGIONS, box=(0, 450, 0, 560)) -> f(x, y) -> bool`

Tạo lưới mọi pixel nguyên trong `box` (`np.meshgrid(..., indexing="ij")`), đổi sang toạ độ hand bằng `pixel2hand`, gọi `hand_surface.locate` để lấy chỉ số vùng của từng điểm, rồi tạo mảng boolean `inside` đánh dấu các điểm có chỉ số thuộc một trong các `tags` (tra bằng `hand_`

`surface.tags.index`). Việc này chỉ làm một lần. Hàm trả về `f(x, y)`: làm tròn (x, y) về pixel nguyên, trả `False` nếu ngoài `box`, ngược lại trả giá trị `inside` tương ứng.

3.5 `build_stimulus`

`build_stimulus(ts, tensor, spec, center_px, duration_s=None, ramp_s=None, fs=5000) -> Stimulus`

```
def build_stimulus(ts, tensor, spec: HDSSpec, center_px, duration_s=None, ramp_s=None, fs=5000):
   duration_s = spec.duration_s if duration_s is None else duration_s
   ramp_s = spec.ramp_s if ramp_s is None else ramp_s
   hand = pins_hand_mm(ts, spec, center_px)
   pin_radius_ts = spec.pin_diameter_mm / 2 / hand_scale(ts) # real mm -> TouchSim mm
   stim = None
   for r in range(tensor.shape[0]):
      for c in range(tensor.shape[1]):
         if not tensor[r, c]:
            continue
         s = ts.stim_ramp(amp=spec.depth_mm, len=duration_s, loc=hand[r, c],
                     ramp_len=ramp_s, fs=fs, pin_radius=pin_radius_ts)
         if stim is None:
            stim = s
         else:
            stim += s
   if stim is None:
      raise ValueError("Tensor has no active pins.")
   return stim
```

Các bước:

1. `duration_s` và `ramp_s` lấy từ `spec` nếu không truyền.
2. Vị trí pin trong hệ hand: `pins_hand_mm`.
3. Bán kính pin đổi sang đơn vị TouchSim: $r_{ts} = \dfrac{\texttt{pin_diameter_mm}/2}{s_h} \approx 5{,}53$ mm (cùng hệ số chia như vị trí pin).
4. Với mỗi ô của `tensor` bằng 1, gọi `ts.stim_ramp` với `amp=spec.depth_mm`, `len=duration_s`, `loc=` tâm pin, `ramp_len=ramp_s`, `fs`, `pin_radius=`r_{ts}.
5. Cộng dồn các `Stimulus` bằng `+=`.
6. Nếu tensor không có ô nào bằng 1, ném `ValueError("Tensor has no active pins.")`.

`depth_mm` được truyền nguyên giá trị, không chia cho s_h.

3.6 `balance_classes`

`balance_classes(pop, ratio=CLASS_RATIO, seed=0) -> AfferentPopulation`

```
def balance_classes(pop, ratio=CLASS_RATIO, seed=0):
   """Randomly subsample afferents so the class counts follow `ratio` (as many as possible)."""
   cls = np.array(pop.affclass)
   unit = min((cls == k).sum() / w for k, w in ratio.items())
```

```
    rng = np.random.default_rng(seed)
    sel = np.zeros(len(cls), bool)
    for k, w in ratio.items():
        sel[rng.choice(np.where(cls == k)[0], int(round(unit * w)), replace=False)] = True
    return pop[sel]
```

Gọi n_k là số afferent của lớp k và w_k là trọng số trong `ratio`. Mã tính

$$u = \min_k \frac{n_k}{w_k}, \qquad m_k = \mathrm{round}(u\, w_k), \tag{8}$$

rồi với mỗi lớp chọn ngẫu nhiên không lặp m_k chỉ số bằng `np.random.default_rng(seed).choice`. Quần thể trả về là `pop[sel]` với `sel` là mặt nạ boolean, nên thứ tự afferent gốc được giữ. Cùng `seed` cho cùng kết quả.

Ví dụ minh hoạ thuật toán: với $(n_{SA1}, n_{RA}, n_{PC}) = (1171, 2711, 1036)$ và tỉ lệ $2:4:1$, $u = \min(585{,}5;\ 677{,}75;\ 1036) = 585{,}5$, nên $m = (1171,\ 2342,\ 586)$. Lớp bị giới hạn (ở đây là SA1) được giữ nguyên. Nếu một lớp trong `ratio` không có afferent nào thì $u = 0$ và quần thể trả về rỗng.

3.7 `make_population`

`make_population(ts, spec, center_px, margin_mm=20.0, density_multiplier=1.0, class_ratio=CLASS_RATIO, seed=0)`

```
def make_population(ts, spec: HDSSpec, center_px, margin_mm=20.0, density_multiplier=1.0,
                    class_ratio=CLASS_RATIO, seed=0):
    """Afferents (SA1, RA, PC) under / near the HDS box (+ margin), class-balanced.
    Build once per HDS position and reuse for every digit so responses are comparable."""
    pop = ts.affpop_hand(density_multiplier=density_multiplier)
    ppm = px_per_mm(ts)
    loc_px = ts.hand_surface.hand2pixel(pop.location)
    x0, y0, x1, y1 = bounding_box_px(spec, center_px, ppm)
    m = margin_mm * ppm
    keep = ((loc_px[:, 0] >= x0 - m) & (loc_px[:, 0] <= x1 + m) &
            (loc_px[:, 1] >= y0 - m) & (loc_px[:, 1] <= y1 + m))
    if not keep.any():
        raise ValueError("No afferents near the HDS - is the center on the hand?")
    pop = pop[keep]
    return balance_classes(pop, class_ratio, seed) if class_ratio else pop
```

Các bước:

1. `ts.affpop_hand(density_multiplier)` tạo quần thể afferent của bàn tay.
2. Đổi vị trí afferent sang pixel bằng `hand2pixel`.
3. Tính hộp HDS bằng `bounding_box_px` với $\rho =$ `px_per_mm(ts)`, mở rộng mỗi phía `margin_mm`$\cdot\rho$ pixel.
4. Giữ afferent có vị trí pixel nằm trong hộp mở rộng (so sánh $\geq, \leq$ cho cả hai trục).
5. Nếu không còn afferent nào, ném `ValueError`.
6. Nếu `class_ratio` khác `None`, gọi `balance_classes`; `class_ratio=None` bỏ qua bước này.

Quần thể được tạo một lần cho một vị trí HDS rồi dùng chung cho cả 10 chữ số.

3.8 `respond`

`respond(ts, stim, pop)`

Trả về `pop.response(stim)`; kết quả có thuộc tính `spikes` (một mảng thời điểm spike cho mỗi afferent).

4 Module `evaluate.py`

`spike_counts(resp) -> ndarray int (N,)`

`np.array([len(s) for s in resp.spikes])`: số spike của từng afferent.

`evaluate(counts_a, counts_b) -> float`

Hệ số Pearson giữa hai vector số spike:

$$r = \frac{\mathrm{cov}(a, b)}{\sigma_a \sigma_b}. \tag{9}$$

Trả về `0.0` nếu một trong hai vector có độ lệch chuẩn bằng 0 (tránh chia cho 0).

`similarity_matrix(all_counts: dict) -> ndarray (n, n)`

Sắp xếp các khoá của `all_counts`, rồi $S_{ab} =$ `evaluate`(c_a, c_b) cho mọi cặp. Với 10 chữ số, ma trận 10×10 đối xứng, đường chéo bằng 1 (khi vector không hằng).

`counts_by_class(counts, pop) -> dict`

Tổng số spike của các afferent thuộc mỗi lớp `"SA1"`, `"RA"`, `"PC"`, dạng `int`, dựa vào `pop.affclass`.

5 Module `viz.py`

`plot_layout(ts, center_px, digit=3, spec=None, highlight=PALM_REGIONS, ax=None) -> ax`

Vẽ trong hệ mm thật (toạ độ pixel chia cho ρ):

1. mỗi đa giác trong `hand_surface.boundary`, tô màu nếu tên vùng thuộc `highlight`;
2. hộp HDS (`bounding_box_px`);
3. 15 vòng tròn bán kính `pin_diameter_mm/2`, tô đỏ ở các pin hoạt động của `digit`.

Giới hạn trục lấy từ đường viền tay cộng 5 mm, tỷ lệ trục bằng nhau.

6 Script `scripts/run_pipeline.py`

`run(ts, density=1.0, step_px=4, search_box=(40, 400, 100, 520)) -> (center, pop, all_counts, S)`

Thực hiện theo thứ tự:

1. In chiều dài tay TouchSim, ρ_{ts}, s_h và ρ.
2. `fit_to_region` với `region_mask_fn(ts)` và $\rho =$ `px_per_mm(ts)`; đổi kết quả thành `center` (tuple hai `float`); in số pin trong sáu vùng lòng bàn tay.
3. `make_population(ts, spec, center, density_multiplier=density)`.
4. Với $d = 0..9$: `build_stimulus` → `respond` → `spike_counts`; in số pin, tổng spike và `counts_by_class`.
5. `similarity_matrix(all_counts)`.

`main()`

Đọc tham số dòng lệnh `--density` (mặc định 1.0) và `--out` (mặc định `out/results.npz`), gọi `run(load_touchsim(), ...)`, in ma trận S, tạo thư mục đầu ra nếu cần và lưu bằng `np.savez` với các khoá: `center`, `similarity`, `affclass`, `counts_0` …`counts_9`.

7 Kiểm thử

Chạy bằng `pytest`; `conftest.py` thêm thư mục gốc và `tests/` vào `sys.path`. Tổng cộng 13 test.

7.1 `tests/test_hds.py` (không cần TouchSim)

Test	Kiểm tra
`test_digits_shape_and_binary`	mỗi tensor có shape $(5, 3)$, giá trị $\subseteq \{0, 1\}$; chữ số 8 có 13 pin, chữ số 0 có 12 pin
`test_geometry_matches_spec`	pitch 15 theo cả hai trục; hộp đúng 50×80; mép ngoài của pin xa nhất nằm trong hộp
`test_scale_applies`	với $\rho = 2$, pitch pixel bằng 30
`test_fit_finds_inside_position`	với vùng hình chữ nhật, `fit_to_region` tìm được vị trí có 15/15 pin trong vùng
`test_timing`	`ramp_s`= 0,2, `duration_s`= 3,0
`test_hamming_symmetric_zero_diagonal`	ma trận Hamming đối xứng, đường chéo bằng 0

7.2 `tests/test_adapter.py`

Test	Kiểm tra
`test_hand_scale_and_ppm`	$L_{ts} = 209$, $s_h = 189/209$, $\rho = \rho_{ts}/s_h$
`test_hds_is_not_scaled_with_hand_in_pixels`	pitch trong hệ hand bằng $15/s_h$; đổi về mm thật (pixel/ρ) bằng đúng 15
`test_pin_radius_follows_position_scaling`	bán kính truyền cho `stim_ramp` bằng $5/s_h$
`test_stimulus_has_one_pin_per_active_cell`	với mọi chữ số, số pin trong kích thích bằng số ô 1; tensor rỗng ném `ValueError`
`test_balance_classes_ratio`	sau cân bằng, $N_{SA1} \approx 2N_{PC}$, $N_{RA} \approx 4N_{PC}$; cùng seed cho cùng quần thể
`test_population_near_hds_only`	mọi afferent nằm trong hộp HDS mở rộng 20 mm
`test_fit_inside_palm_and_pipeline_smoke`	`fit_to_region` với `region_mask_fn` có kết quả hợp lệ; chạy 10 chữ số tạo ma trận 10×10 đối xứng, đường chéo bằng 1

7.3 `tests/fake_touchsim.py`

Module thay thế `touchsim` tối thiểu để chạy test của `adapter.py` mà không cần cài TouchSim. Nó cung cấp: `hand_surface` với `pxl_per_mm`= 2,18, bảy hình chữ nhật (sáu vùng lòng bàn tay và một vùng ngón giữa) sao cho chiều dài đường viền là 209 mm, `pixel2hand`/`hand2pixel` tuyến tính, `locate`; `stim_ramp` trả về đối tượng lưu danh sách (vị trí, bán kính) và hỗ trợ `+=`; `affpop_hand` sinh quần thể ngẫu nhiên có seed với tỉ lệ lớp xấp xỉ 24/55/21%; `response` sinh số spike theo Poisson giảm dần theo khoảng cách tới pin gần nhất. Vì vị trí afferent và số spike là sinh giả, các test này kiểm tra logic của mã (đơn vị, chỉ số, luồng dữ liệu), không kiểm tra đầu ra của TouchSim thật.

8 Notebook `hds_touchsim_clean.ipynb`

Notebook chứa cùng mã trong một tệp, theo thứ tự các cell: (1) cài đặt và import; (2) mã của `hds.py`; (3) các test của `test_hds.py`, chạy ngay trong notebook; (4) mã của `adapter.py`; (5) mã của `evaluate.py` và `viz.py`; (6) đặt HDS (`fit_to_region`), tạo quần thể và vẽ `plot_layout`; (7) mô phỏng chữ số 0–9; (8) in ma trận S và hệ số tương quan giữa H và S trên tam giác trên (`np.triu_indices(10, 1)`).

9 Ghi chú về hành vi của mã

- `pressure_mpa` chỉ là trường của `HDSSpec`, không hàm nào dùng.
- Vị trí và bán kính pin được chia cho s_h khi gửi sang TouchSim; `depth_mm` thì không.
- L_{ts} (và do đó s_h) được định nghĩa bằng chính phép đo `ptp` theo y của toàn bộ đường viền, nên chiều dài tay sau khi thu luôn bằng `REAL_HAND_LENGTH_MM` theo cấu tạo.
- `region_mask_fn` trả `False` cho điểm nằm ngoài `box`.

- `make_population` loại afferent ngoài hộp HDS mở rộng `margin_mm`.
- `balance_classes` trả quần thể rỗng nếu một lớp trong `ratio` không có afferent.
- `evaluate` trả 0 (không phải `nan`) khi một vector spike không đổi.

References

[1] H. P. Saal, B. P. Delhaye, B. C. Rayhaun, S. J. Bensmaia. *Simulating tactile signals from the whole hand with millisecond precision.* PNAS, 114(28):E5693–E5702, 2017.